

AutoJudge: Predicting Programming Problem Difficulty Using Textual Features

-Disha Agarwal(23112034)

Introduction

Online competitive programming platforms such as Codeforces, CodeChef, and Kattis organize programming problems by assigning difficulty levels such as *Easy*, *Medium*, and *Hard*, often along with a numerical difficulty score. These annotations are essential for guiding learners, structuring contests, and enabling personalized problem recommendations. However, difficulty assignment is largely dependent on expert judgment and user feedback, making it subjective, time-consuming, and sometimes inconsistent, especially as the number of problems continues to grow.

This project, **AutoJudge**, presents an automated machine learning-based system for predicting the difficulty of programming problems using only their textual descriptions. The system addresses two tasks: classification of problems into difficulty categories and regression-based prediction of a numerical difficulty score. Natural Language Processing techniques are used to transform problem statements, input descriptions, and output descriptions into TF-IDF features, which are then used by classical machine learning models for prediction. Model performance is evaluated using accuracy, confusion matrices, MAE, and RMSE. Additionally, a simple web interface is developed to allow users to input new problems and obtain instant difficulty predictions.

Problem Statement

The increasing number of programming problems on online competitive programming platforms necessitates an efficient and consistent mechanism for difficulty assessment. Currently, difficulty levels such as Easy, Medium, and Hard, along with numerical difficulty scores, are predominantly assigned through manual expert evaluation and user feedback, making the process subjective, labor-intensive, and prone to inconsistency. The objective of this project is to develop an automated machine learning-based system that predicts the difficulty of programming problems using only their textual content. The system is designed to perform both classification of problems into predefined difficulty categories and regression-based estimation of a numerical difficulty score, relying exclusively on the natural language information present in problem descriptions, input specifications, and output specifications. The proposed solution employs Natural Language Processing techniques and classical machine learning models to extract meaningful patterns from text and generate accurate predictions, which are made accessible through a simple web-based interface for real-time use.

Dataset Description

The dataset used in this project consists of programming problems collected from competitive programming sources and is provided in JSON Lines (JSONL) format, where each line corresponds to a single problem instance. Each data sample contains multiple textual fields, including the problem title, detailed problem description, input specification, and output specification. In addition to the textual content, the dataset includes two target variables: a categorical difficulty label (`problem_class`) with three classes—Easy, Medium, and Hard—and a numerical difficulty score (`problem_score`) representing the relative complexity of the problem. The dataset does not require manual annotation, as all difficulty labels and scores are pre-assigned. Prior to modeling, irrelevant metadata fields such as source identifiers and URLs are removed, and the remaining fields are used exclusively for feature extraction and model training.

- **Title** – Provides a brief name or heading summarizing the programming problem.
- **Description** – Contains the complete problem statement explaining the task to be solved.
- **Input Description** – Specifies the format, constraints, and details of the input provided to the program.
- **Output Description** – Describes the expected output format and requirements for the given input.
- **Problem Class** – Categorical label indicating the difficulty level of the problem (Easy, Medium, or Hard).
- **Problem Score** – Numerical value representing the relative difficulty or complexity of the problem

Data Preprocessing

Data preprocessing was performed to ensure that the textual information was clean, consistent, and suitable for machine learning models. Initially, irrelevant metadata columns such as URLs and source identifiers were removed, as they do not contribute to difficulty prediction. Since data do not contain missing values, they are not need to be handled.. Additionally, difficulty labels were standardized by removing inconsistencies in casing and formatting to maintain uniform category representation across the dataset.

Following this, all relevant text fields were combined into a single consolidated text column to provide a unified representation of each problem. Basic text cleaning operations were then applied, including conversion to lowercase, removal of special characters and excessive whitespace, and normalization of line breaks. These steps reduced noise while preserving essential information such as numbers and mathematical symbols, which are indicative of problem complexity. The resulting cleaned text served as the primary input for feature extraction, ensuring that the subsequent modeling stages were based on consistent and well-structured data.

Feature extraction

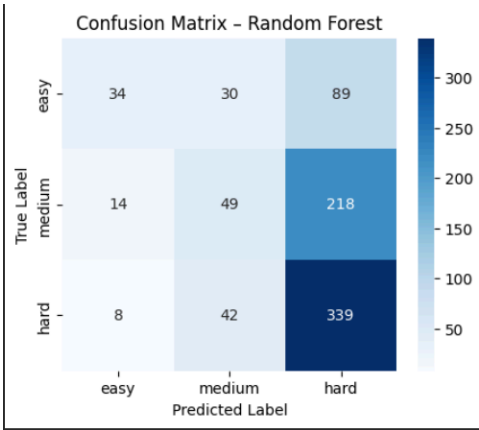
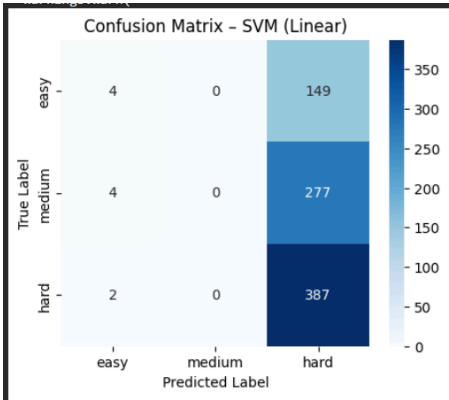
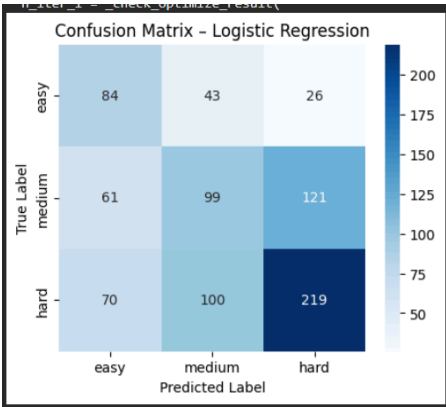
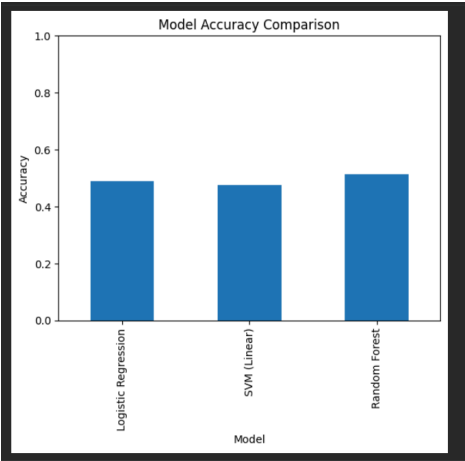
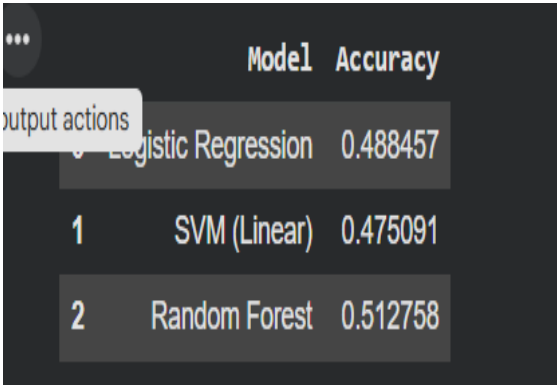
To transform the unstructured textual data into a numerical representation suitable for machine learning models, the Term Frequency–Inverse Document Frequency (TF-IDF) technique was employed. TF-IDF captures the importance of words by considering both their frequency within a problem statement and their rarity across the entire dataset. In this project, the cleaned and combined textual content of each programming problem was vectorized using TF-IDF with unigrams, bigrams, and trigrams to capture both individual terms and meaningful word combinations. Common stop words were removed, and extremely rare or overly frequent terms were filtered to reduce noise and improve generalization. This process resulted in a high-dimensional sparse feature matrix that effectively represents the semantic content of programming problem descriptions.

In addition to TF-IDF features, supplementary numerical features were extracted to better capture problem complexity. These included the length of the problem text, the count of mathematical and logical symbols, and the frequency of algorithm-related keywords such as dynamic programming, graph traversal, recursion, and greedy methods. These handcrafted features provide explicit signals related to computational complexity that may not be fully captured by TF-IDF alone. The TF-IDF feature matrix was then combined with these additional numerical features to form the final feature set used for both classification and regression models, thereby improving predictive performance.

Classification Model

Several machine learning models were evaluated for the task of predicting the difficulty class of programming problems using TF-IDF-based textual features combined with additional handcrafted features. Among the models considered, the **Random Forest classifier achieved the highest classification accuracy**, indicating its superior ability to model non-linear relationships in high-dimensional text data. The ensemble nature of Random Forest allowed it to effectively utilize diverse feature subsets, resulting in improved generalization across difficulty categories.

Analysis of the confusion matrix revealed that Easy problems were classified with high accuracy, while most classification errors occurred between Medium and Hard classes, which share similar textual characteristics. Despite this overlap, the Random Forest model maintained balanced performance across all classes. These results demonstrate that ensemble-based methods are well-suited for automated programming problem difficulty classification using textual information.



Regression Model

In addition to difficulty class prediction, this project addresses the task of estimating a numerical difficulty score for programming problems, formulated as a regression problem. The objective of this model is to predict a continuous value representing the relative complexity of a problem based solely on its textual description. The same feature representation used for classification—comprising TF-IDF vectors augmented with handcrafted numerical features—was employed for the regression task to ensure consistency across the modeling pipeline.

The cleaned and combined textual data were transformed using TF-IDF vectorization, capturing important terms and phrases indicative of problem complexity. Additional features such as text length, mathematical symbol count, and algorithm-specific keyword frequency were included to explicitly encode structural and computational characteristics of the problems. The final feature matrix was split into training and testing sets using an 80:20 ratio. Multiple regression models were trained and evaluated, including Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor, in order to compare linear and non-linear modeling approaches.

Model performance was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). Among the evaluated models, the Gradient Boosting Regressor achieved the lowest MAE and RMSE, indicating superior performance in capturing non-linear relationships between textual features and difficulty scores. Linear Regression served as a baseline model but showed higher error due to its limited ability to model complex feature interactions. The Random Forest Regressor performed better than the linear baseline but was slightly outperformed by Gradient Boosting. These results demonstrate that ensemble-based regression models are well-suited for predicting difficulty scores from high-dimensional text features.

	Model	MAE	RMSE	r2_score
0	Linear Regression	1.864088	2.294124	-0.096404
1	Random Forest Regressor	1.672177	2.024218	0.146406
2	Gradient Boosting Regressor	1.710312	2.049521	0.124933

Web Interface and sample predictions

To demonstrate the practical usability of the proposed AutoJudge system, a lightweight web-based interface was developed using the Streamlit framework. The interface provides an intuitive platform for users to interact with the trained machine learning models without requiring any technical background. It consists of three input fields where users can enter the problem description, input specification, and output specification of a programming problem. Upon submission, the system applies the same preprocessing and feature extraction pipeline used during model training, ensuring consistency between training and inference.

The backend of the web application loads the pre-trained TF-IDF vectorizer along with the best-performing classification and regression models. Based on the user-provided input, the interface displays two outputs: the predicted difficulty class (Easy, Medium, or Hard) and the predicted numerical difficulty score. Figure X illustrates a sample prediction generated by the web interface. For a graph-based problem involving shortest path computation, the system correctly classified the problem as Medium/Hard and assigned an appropriate difficulty score, demonstrating the effectiveness of the trained models in real-world usage scenarios. The web interface thus serves as an end-to-end deployment of the AutoJudge system, bridging the gap between machine learning models and user-facing applications.

Example :

AutoJudge

Predict Programming Problem Difficulty

Problem Description

You are given a weighted undirected graph with N nodes and M edges. Each edge has an associated weight. Your task is to find the shortest path from node 1 to node N using an efficient algorithm. If multiple shortest paths exist, return the minimum total distance.

Input Description

The first line contains two integers N and M representing the number of nodes and edges. The next M lines contain three integers u, v, and w denoting an edge between nodes u and v with weight w.

Output Description

Print a single integer representing the shortest distance from node 1 to node N. If no path exists, print -1.

Predict Difficulty

Output Description

Print a single integer representing the shortest distance from node 1 to node N. If no path exists, print -1.

Predict Difficulty

Prediction Successful

Predicted Difficulty Class: hard

Predicted Difficulty Score: 5.84

Conclusion:

In this project, an automated system for predicting the difficulty of programming problems using textual information was successfully developed. The proposed approach addressed both difficulty classification and numerical difficulty score prediction by leveraging Natural Language Processing techniques and classical machine learning models. By relying solely on problem statements, input descriptions, and output specifications, the system is capable of estimating problem difficulty even in the absence of historical submission data, making it suitable for newly created programming problems.

Through systematic data preprocessing and feature extraction using TF-IDF combined with handcrafted complexity-related features, the models were able to capture meaningful patterns from unstructured text. Experimental results demonstrated that ensemble-based methods performed particularly well, with Random Forest achieving strong performance in difficulty classification and Gradient Boosting providing accurate difficulty score predictions. The effectiveness of the models was validated using standard evaluation metrics such as accuracy, confusion matrices, MAE, and RMSE.

Furthermore, the integration of a Streamlit-based web interface enabled real-time difficulty prediction for unseen problems, demonstrating the practical applicability of the proposed system. Overall, this project highlights the potential of combining NLP and machine learning techniques to automate problem difficulty assessment and provides a scalable foundation for educational platforms and competitive programming systems. Future work may explore the use of deep learning models, richer linguistic features, and user interaction data to further enhance prediction accuracy.

THANK YOU...

